

HomePlanner Project Report

1. Executive Summary

HomePlanner is a full-stack web application for designing residential floor plans in a browser. The application lets users register, sign in, create projects, draw 2D floor plans, add architectural elements such as walls, doors, double doors, windows, gates, stairs, furniture, and text labels, edit precise properties, and view the same plan in a live 3D scene.

The project demonstrates practical full-stack engineering skills across frontend development, backend API design, authentication, state management, canvas-based editing, 3D visualization, persistence, UI/UX design, and deployment readiness.

From a job-search perspective, this is stronger than a basic CRUD app because it includes:

- Real interactive graphics.
- Persistent user-owned projects.
- Authentication and protected routes.
- Complex client-side state.
- 2D geometry and 3D rendering.
- Export, save, undo, redo, keyboard shortcuts, and property editing.
- A polished dashboard and authentication experience.
- Docker-based local development.

This project can be positioned as a portfolio-level product prototype for roles involving frontend engineering, full-stack engineering, interactive web applications, SaaS tools, or visualization-heavy applications.

2. Problem Statement

Many floor-planning tools are either too simple, too expensive, or require installation. HomePlanner solves this by providing a browser-based planning tool where a user can:

1. Create an account.
2. Manage multiple design projects.
3. Draw a floor plan directly in 2D.
4. Add construction elements and furniture.
5. Edit exact properties.
6. View the result in 3D.
7. Save progress automatically.

The main technical challenge is that this is not only a form-based application. The editor needs to behave like a design tool. It must handle drawing, snapping, dragging, resizing,

rotating, keyboard shortcuts, object selection, unit conversions, and live rendering.

3. What Has Been Built

3.1 Authentication

The app includes user authentication with:

- Register page.
- Login page.
- JWT-based authentication.
- Persisted auth state on the frontend.
- Protected editor/dashboard routes.
- Automatic logout handling when a token becomes invalid.

Relevant files:

- frontend/src/pages/LoginPage.tsx
- frontend/src/pages/RegisterPage.tsx
- frontend/src/store/useAuthStore.ts
- frontend/src/api/client.ts
- backend/app/api/v1/auth.py
- backend/app/core/security.py
- backend/app/models/user.py

Recent UI improvements:

- Added a high-quality architecture-themed background image.
- Converted login/register forms into polished glass-style cards.
- Centered the HomePlanner branding above the form.
- Added icon-enhanced form fields for email, username, and password.

3.2 Dashboard

The dashboard lets the user manage projects.

Implemented capabilities:

- View all projects owned by the logged-in user.
- Create a new project.
- Delete an existing project.
- Open projects.
- Double-click anywhere on a project card to open it.
- Show project metadata:
 - project name

- description
- wall count
- object count
- updated date
- Aesthetic background image.
- Restyled project cards and buttons.

Relevant files:

- frontend/src/pages/DashboardPage.tsx
- frontend/src/assets/dashboard-home-bg.png
- frontend/src/utls/projectThumbnails.ts
- backend/app/api/v1/projects.py

Important implementation note:

Project thumbnails are currently stored in browser localStorage after capturing the 2D canvas during save or auto-save. This works for local use, but for production it should be moved to backend storage so previews work across devices.

3.3 Project Management

Users can create, load, update, and delete projects. Each project stores a full floor plan.

The backend exposes endpoints for:

- create project
- list projects
- get project detail
- update project metadata
- save floor plan
- delete project

Relevant files:

- backend/app/api/v1/projects.py
- backend/app/models/project.py

The project data model includes:

- walls
- openings
- objects
- canvas size
- grid size

3.4 2D Floor Plan Editor

The 2D editor is the core user-facing feature.

Implemented editor capabilities:

- Draw walls.
- Snap walls to grid.
- Snap walls to nearby wall endpoints.
- Hold Shift for straight wall drawing.
- Select and edit walls.
- Drag wall endpoints.
- Move connected wall endpoints.
- Draw and place doors.
- Draw and place double doors.
- Add windows.
- Add gates.
- Add stairs.
- Add furniture.
- Add text labels.
- Pan and zoom the canvas.
- Scroll large floor plans.
- Delete selected elements.
- Edit text directly.
- Export the 2D plan image.
- Capture thumbnails for dashboard previews.

Relevant file:

- frontend/src/components/editor/Canvas2D.tsx

3.5 Door and Window Improvements

Several architectural element improvements were added:

- Combined door tools into a parent Door menu.
- Door menu reveals:
 - Single Door
 - Double Door
- Double Door remains its own placement mode.
- Window 2D color changed to black.
- Removed window length text from the 2D canvas.
- Door and window placement previews remain interactive.

Relevant files:

- frontend/src/components/layout/Toolbar.tsx
- frontend/src/components/editor/Canvas2D.tsx

3.6 Furniture Sketch Mode

The 2D furniture rendering was changed from colored filled shapes to a basic black sketch style.

Implemented changes:

- Furniture now appears as clean black-outline symbols.
- Removed furniture names from the 2D canvas.
- Kept text objects visible.
- Kept furniture selection, dragging, resizing, and rotation behavior.

This improves the 2D plan readability because the canvas now looks more like a technical sketch instead of a decorative object layout.

Relevant file:

- frontend/src/components/editor/Canvas2D.tsx

3.7 Properties Panel

The right-side Properties panel allows exact editing of selected elements.

Wall properties include:

- start X
- start Y
- end X
- end Y
- editable length
- thickness
- height
- 3D wall color

Opening properties include:

- width
- position percentage
- door style
- height
- elevation
- trim
- swing angle
- rotation
- mount position
- swing side
- swing direction
- hardware style
- frame color
- panel color

Object properties include:

- X
- Y
- width
- depth/height
- rotation
- color
- label

Recent improvements:

- Wall height now correctly affects actual 3D wall height.
- Wall length is editable directly.
- Length input adapts to the selected unit:
 - meters
 - feet
 - feet/inches notation
- Inches mode supports values like 5' 6".
- Added up/down chevrons for inch stepping.
- Replaced the properties panel toggle icon with left/right arrows.

Relevant file:

- frontend/src/components/layout/PropertiesPanel.tsx

3.8 Measurement Units

The application supports multiple length display modes:

- meters
- feet
- feet/inches

Implemented behavior:

- Wall length labels can display in the selected unit.
- The properties panel length field changes based on the selected unit.
- Feet/inches mode uses architectural notation such as 5' 6".

Relevant files:

- frontend/src/components/layout/EditorHeader.tsx
- frontend/src/components/layout/PropertiesPanel.tsx
- frontend/src/components/editor/Canvas2D.tsx

3.9 Undo and Redo

The app supports editing history:

- Undo with Ctrl+Z.
- Redo with Ctrl+Y.
- Undo and redo buttons in the toolbar.
- History snapshots stored in Zustand.

Relevant files:

- frontend/src/store/useFloorPlanStore.ts
- frontend/src/components/editor/Canvas2D.tsx
- frontend/src/components/layout/Toolbar.tsx

3.10 File Menu

A File menu was added near the Dashboard button.

Menu actions include:

- New project
- Open folder
- Save
- Save as
- Export image
- Print
- Record 3D video

Keyboard shortcuts were added for file-style actions:

- Ctrl+N
- Ctrl+S
- Ctrl+Shift+S
- Ctrl+P

Relevant file:

- frontend/src/components/layout/EditorHeader.tsx

3.11 3D Viewer

The 3D viewer renders the floor plan in a 3D scene.

Implemented 3D capabilities:

- Walls rendered as 3D geometry.
- Wall height from properties affects 3D height.
- Door frames and door leaves.
- Double door rendering.
- Gate rendering.
- Window frame rendering.
- Furniture rendered as basic 3D objects.

- Wood floor texture.
- Orbit controls.
- Zoom controls.
- Pan controls.
- Split view support.
- 3D video recording support.

Relevant file:

- frontend/src/components/viewer3d/Viewer3D.tsx

3.12 Save, Auto-Save, Export, and Print

The project supports persistence and output workflows.

Implemented:

- Manual save.
- Debounced auto-save.
- Save as PDF.
- Save as Word document.
- Save as image.
- Print plan.
- 2D canvas export.
- 3D video recording.
- Dashboard thumbnail capture.

Relevant files:

- frontend/src/components/layout/EditorHeader.tsx
- frontend/src/hooks/useAutoSave.ts
- frontend/src/components/editor/Canvas2D.tsx
- frontend/src/utils/projectThumbnails.ts

4. Technology Stack

4.1 Frontend

The frontend uses:

- React 18
- TypeScript
- Vite
- Tailwind CSS
- Zustand

- Axios
- Konva / React Konva
- Three.js
- React Three Fiber
- Drei
- Lucide React icons

Why these choices matter:

- React provides component-driven UI.
- TypeScript improves safety in complex state and geometry logic.
- Vite gives fast development and simple production builds.
- Zustand keeps editor state simple without Redux boilerplate.
- Konva is appropriate for 2D canvas editing.
- Three.js and React Three Fiber are appropriate for browser 3D visualization.
- Axios centralizes API calls and token handling.

4.2 Backend

The backend uses:

- FastAPI
- Python 3.11
- MongoDB
- Beanie ODM
- Pydantic models
- JWT authentication
- Password hashing

Why these choices matter:

- FastAPI is well-suited for typed REST APIs.
- Pydantic gives schema validation.
- MongoDB fits nested floor-plan documents.
- Beanie provides document modeling on top of MongoDB.
- JWT allows stateless authentication.

4.3 Infrastructure

The project includes Docker support:

- docker-compose.yml
- backend Dockerfile
- frontend Dockerfile
- MongoDB service

For local development, the app can be started with:

```
"bash
docker-compose up --build
'
```

5. Architecture

5.1 High-Level Architecture

The architecture is:

```
'text
```

Browser

React frontend

- Dashboard
- 2D editor
- 3D viewer
- Auth pages

V

FastAPI backend

REST API

JWT auth

Project persistence

V

MongoDB database

5.2 Frontend Architecture

The frontend separates concerns as follows:

- pages/: route-level screens.
- components/editor/: 2D canvas editor.
- components/viewer3d/: 3D scene.
- components/layout/: toolbar, header, status bar, properties panel.
- store/: Zustand state stores.
- api/: API client and error handling.
- types/: shared TypeScript schema definitions.
- utils/: reusable utilities such as thumbnail capture.

5.3 Backend Architecture

The backend separates concerns as follows:

- api/v1/: route handlers.
- models/: database and request/response schemas.
- core/: config, database connection, security.
- main.py: app creation, middleware, router registration.

5.4 Data Flow

Typical project workflow:

1. User logs in.
2. Frontend stores JWT.
3. User opens dashboard.
4. Frontend requests project list.
5. User opens a project.
6. Frontend loads full floor plan.
7. Zustand hydrates editor state.
8. User edits the floor plan.
9. Zustand updates local state.
10. Auto-save sends floor plan to backend.
11. Backend validates and stores the floor plan in MongoDB.
12. 2D and 3D views update from the same state.

6. Implementation Details

6.1 Authentication Implementation

The backend handles registration and login. Passwords are hashed before storage. Login returns a JWT access token. The frontend stores the token and sends it with API requests.

Frontend API behavior:

- Axios attaches the token to outgoing requests.
- If an API call returns 401, token data is removed and the user is redirected to login.

This demonstrates:

- Authentication flow.
- Token-based API security.
- Protected frontend routes.
- Basic session persistence.

6.2 Floor Plan Data Model

A floor plan is stored as a nested document:

```
'text
FloorPlan
walls[]
openings[]
objects[]
canvas_width
canvas_height
grid_size
'
```

Each wall has:

- start point
- end point
- thickness
- height
- optional color

Each opening has:

- wall reference
- type
- offset along wall
- width
- swing information
- 3D styling properties

Each object has:

- type
- label
- position
- width
- height
- rotation
- color

This model is suitable for MongoDB because the floor plan is naturally document-shaped. A project owns a complete floor plan, and saving the entire floor plan as one document is simple and practical for this app stage.

6.3 2D Editor Implementation

The 2D editor is built with React Konva. The main canvas state comes from Zustand.

Core editor concepts:

- Canvas coordinates are stored in pixels.
- Walls are line segments.
- Openings are attached to a wall by wall_id and an offset.
- Furniture objects are positioned using center coordinates.
- Snapping is applied during placement.
- Selection controls determine what appears in the Properties panel.

Important interaction logic:

- Mouse down starts wall drawing or placement.
- Mouse move updates previews.
- Double-click ends wall drawing.
- Dragging updates objects and wall endpoints.
- Keyboard events switch tools and trigger undo/redo.
- Export events generate canvas images.

This is one of the most technically valuable parts of the project because it shows custom interaction programming beyond standard forms and tables.

6.4 Snapping and Geometry

The editor includes geometry helpers for:

- snapping to grid
- snapping to wall endpoints
- projecting points onto walls
- finding closest wall
- computing wall length
- computing wall angle
- positioning openings along walls

These functions are needed because the editor is spatial. User actions must map from pointer coordinates to floor-plan geometry.

This demonstrates practical math in UI development.

6.5 3D Viewer Implementation

The 3D viewer uses Three.js through React Three Fiber.

The app converts 2D floor-plan data into 3D objects:

- wall line segments become 3D boxes
- wall thickness becomes box depth
- wall height becomes box height
- wall position maps into the 3D X/Z plane
- openings create wall gaps
- doors/windows/gates get additional 3D geometry

- furniture gets simplified 3D object forms

The conversion uses a scale:

'text

50px = 1 meter

,

This allows the same stored 2D data to drive both the 2D editor and 3D scene.

6.6 State Management

Zustand stores:

- walls
- openings
- objects
- selected element
- active tool
- measurement unit
- wall measurement mode
- dirty state
- undo/redo history

This is appropriate because editor state is shared by many components:

- Canvas2D
- Viewer3D
- PropertiesPanel
- Toolbar
- StatusBar
- EditorHeader

Using a central store avoids excessive prop drilling.

6.7 Undo/Redo Implementation

Undo/redo is implemented by storing snapshots of the floor plan.

When a significant edit starts, the app pushes a snapshot. Undo restores a previous snapshot. Redo reapplies a future snapshot.

This approach is simple and appropriate for a project of this size. For very large plans, a future improvement would be command-based history instead of full snapshots.

6.8 Auto-Save Implementation

Auto-save is debounced.

Behavior:

1. User edits the plan.
2. Store marks state as dirty.
3. useAutoSave waits briefly.
4. If the user stops editing, the floor plan is sent to backend.
5. On success, state is marked clean.

This prevents saving on every tiny movement while still keeping user work safe.

6.9 Dashboard Thumbnail Implementation

The app captures a 2D canvas image and stores it in localStorage as a project thumbnail.

Current behavior:

- Capture happens during save/auto-save.
- Dashboard reads thumbnails from localStorage.
- If no thumbnail exists, a fallback preview is shown.

Limitation:

- localStorage thumbnails are device-specific.

Production improvement:

- Store thumbnail on backend as part of the project model, or upload it to object storage.

6.10 File Export Implementation

The app supports exporting the plan as:

- image
- PDF
- Word document

It captures the 2D canvas, converts it into image data, and wraps it into the selected output format.

This demonstrates:

- browser canvas export
- Blob handling
- file download flow
- print workflow

7. Design and UX Improvements

Several UI improvements were made to move the app from functional prototype toward a polished product:

- Aesthetic dashboard background.
- Aesthetic login/register background.
- Better auth form card.
- Centered auth branding.
- File menu with familiar actions.
- Collapsible properties panel with arrow icon.
- Door submenu with Single Door and Double Door.
- Cleaner project cards.
- Double-click project opening.
- More subtle dashboard action buttons.
- Black sketch-style 2D furniture.
- Removed visual clutter such as furniture labels and window length text.

These changes matter because employers will evaluate not only whether the app works, but also whether it feels usable and professionally considered.

8. What You Gained From This Project

8.1 Frontend Engineering Skills

You gained experience with:

- React component architecture.
- TypeScript types and interfaces.
- Complex state management with Zustand.
- Canvas-based interaction using Konva.
- Keyboard shortcuts.
- Form handling.
- Responsive UI layouts.
- Styling with Tailwind CSS.
- UI refinement and product polish.
- File export and browser APIs.
- Image assets and visual design.

8.2 Backend Engineering Skills

You gained experience with:

- FastAPI route design.
- Pydantic models.
- MongoDB document modeling.

- Beanie ODM.
- JWT authentication.
- Password hashing.
- Environment-based configuration.
- REST API design.
- Dockerized development.

8.3 Full-Stack Skills

You gained experience connecting frontend and backend:

- Axios API client.
- Token injection into requests.
- Protected routes.
- Persisted user sessions.
- Project CRUD.
- Auto-save to backend.
- Data model shared across frontend and backend.

8.4 Product Engineering Skills

You gained experience thinking like a product engineer:

- Improving usability.
- Reducing visual clutter.
- Adding shortcuts.
- Supporting undo/redo.
- Handling save/export workflows.
- Making dashboard/project management feel practical.
- Balancing technical function with user experience.

8.5 Graphics and Visualization Skills

This project gives you practical experience with:

- 2D geometry.
- Canvas rendering.
- Drag/drop behavior.
- Snapping.
- Spatial data models.
- Converting 2D data into 3D representations.
- Camera/orbit controls.
- 3D scene composition.

This is valuable because many web developers only build CRUD dashboards. This app

demonstrates more advanced interactive UI work.

9. How To Use This Project In Job Search

9.1 Portfolio Positioning

You can present HomePlanner as:

> A full-stack browser-based floor planning application with authenticated project management, a custom 2D canvas editor, real-time 3D visualization, auto-save, export tools, and project dashboard.

This positioning shows:

- full-stack capability
- frontend depth
- graphics/interactions
- product thinking
- real-world architecture

9.2 Resume Bullet Points

Use resume bullets like these:

- Built a full-stack home floor-planning application using React, TypeScript, FastAPI, MongoDB, and Three.js.
- Implemented a custom 2D canvas editor with wall drawing, snapping, selection, dragging, resizing, doors, windows, stairs, furniture, and text annotations.
- Developed real-time 3D visualization by converting 2D floor-plan geometry into Three.js wall, opening, floor, and furniture objects.
- Added authenticated project management with JWT login, protected routes, MongoDB persistence, project CRUD, and debounced auto-save.
- Designed an interactive properties inspector for precise editing of wall dimensions, height, thickness, openings, furniture, and measurement units.
- Implemented undo/redo history, keyboard shortcuts, project thumbnails, file export, print, and save-as workflows.
- Improved UI/UX with polished dashboard, login/register pages, icon-based menus, sketch-style 2D rendering, and responsive editor layouts.
- Dockerized the application for local development with separate frontend, backend, and MongoDB services.

9.3 LinkedIn Project Description

Use this:

> HomePlanner is a full-stack floor-planning web application that allows users to create,

edit, save, and visualize home layouts directly in the browser. I built a custom 2D editor with React Konva, implemented real-time 3D visualization with Three.js/React Three Fiber, and developed a FastAPI + MongoDB backend with JWT authentication and project persistence. The app supports wall drawing, snapping, doors, windows, furniture, stairs, undo/redo, measurement units, auto-save, project dashboard management, export, print, and polished authentication screens.

9.4 GitHub README Pitch

Add a short pitch near the top of the README:

> HomePlanner is a browser-based home design tool built as a full-stack interactive web application. It combines a custom 2D canvas editor with real-time 3D rendering, authenticated project persistence, export workflows, and a polished SaaS-style dashboard.

9.5 Interview Explanation

When an interviewer asks, "Tell me about this project," use this structure:

1. Problem: I wanted to build a browser-based floor planning tool.
2. Stack: React, TypeScript, Konva, Three.js, FastAPI, MongoDB.
3. Frontend challenge: Building a custom editor with drawing, snapping, dragging, and keyboard shortcuts.
4. Backend challenge: Persisting nested floor-plan documents per user with authentication.
5. 3D challenge: Mapping 2D geometry into a live 3D scene.
6. Product work: Added dashboard, auto-save, export, undo/redo, file menu, and polished UI.
7. What I learned: Spatial UI, state management, 3D rendering, full-stack architecture, product polish.

Example answer:

> I built HomePlanner as a full-stack floor-planning app. The frontend is React and TypeScript, with Konva for the 2D editor and React Three Fiber for 3D visualization. The backend is FastAPI with MongoDB and JWT authentication. The hardest part was designing the editor state and geometry model so the same wall, opening, and furniture data could drive both the 2D canvas and the 3D scene. I also implemented snapping, undo/redo, properties editing, auto-save, export, and project management. This project helped me move beyond CRUD apps and work with interactive graphics, spatial data, and real product workflows.

10. Technical Talking Points For Interviews

10.1 Why MongoDB?

MongoDB works well because a floor plan is naturally nested:

- a project has walls
- a project has openings
- a project has objects
- all are saved together

This avoids needing many relational joins for the MVP.

10.2 Why Zustand?

Zustand is lightweight and works well for editor state. The editor needs shared access to walls, openings, objects, active tool, selected item, undo history, and dirty state. Zustand handles that without large Redux boilerplate.

10.3 Why Konva?

Konva provides a React-friendly canvas abstraction for interactive 2D graphics. It supports shapes, drag events, layers, groups, and pointer interactions, which are important for a design editor.

10.4 Why Three.js?

Three.js is the standard browser 3D library. React Three Fiber makes it easier to write 3D scenes using React components.

10.5 What Was The Hardest Part?

The hardest part was keeping the floor-plan data model consistent while supporting multiple views and interactions:

- 2D editing
- 3D rendering
- properties panel
- auto-save
- undo/redo
- export

The same data has to stay valid across all those features.

10.6 How Would You Improve It?

Strong future improvements:

- Store project thumbnails on backend instead of localStorage.
- Add collaborative editing.
- Add sharing links.
- Add custom furniture library.
- Add more realistic 3D materials.

- Add roof/ceiling controls.
- Add room detection and area calculation.
- Add tests for geometry helpers.
- Add production-grade CORS configuration.
- Add cloud deployment pipeline.
- Add project version history.

11. Deployment Plan

The recommended deployment architecture is:

```
'text
```

Frontend: Vercel or Netlify

Backend: Render, Railway, or Fly.io

Database: MongoDB Atlas

```
,
```

11.1 Database

Use MongoDB Atlas.

Set:

```
'env
```

MONGO_URL=mongodb+srv://...

DB_NAME=homeplanner

```
,
```

11.2 Backend

Host FastAPI on Render or Railway.

Production start command:

```
'bash
```

uvicorn app.main:app --host 0.0.0.0 --port \$PORT

```
,
```

Environment variables:

```
'env
```

MONGO_URL=your_mongodb_atlas_url

DB_NAME=homeplanner

SECRET_KEY=your_long_random_secret

ALGORITHM=HS256

ACCESS_TOKEN_EXPIRE_MINUTES=1440

11.3 Frontend

Host Vite frontend on Vercel or Netlify.

Build command:

```
'bash
npm run build
'
```

Output directory:

```
'text
dist
'
```

Frontend environment variable:

```
'env
VITE_API_URL=https://your-backend-url.com
'
```

11.4 Production Security Fixes

Before public release:

- Replace development SECRET_KEY.
- Restrict backend CORS to your frontend domain.
- Remove --reload from backend production command.
- Do not expose MongoDB publicly without authentication.
- Move thumbnails from localStorage to backend storage.

12. Current Limitations

The app is strong for a portfolio project, but these limitations should be clear:

- Project thumbnails are local to one browser.
- No real-time collaboration.
- No sharing permissions.
- No room auto-detection.
- No automated tests yet.
- Docker frontend is development-mode Vite, not production static serving.
- Some 3D geometry is simplified.
- No cloud deployment configuration yet.

These are acceptable for an MVP, but you should be able to explain them.

13. Recommended Next Improvements

Priority order:

1. Store dashboard thumbnails in the backend.
2. Add production deployment configuration.
3. Add tests for geometry helper functions.
4. Add room/area detection.
5. Add project sharing/export links.
6. Improve 3D realism.
7. Add project version history.
8. Add better mobile/tablet responsiveness.

The highest-impact job-search improvement is backend-stored thumbnails plus deployed live URL. A recruiter or interviewer should be able to open the app, create an account, create a project, draw a plan, and see it saved.

14. Suggested Portfolio Page Content

Title

HomePlanner - Full-Stack 2D/3D Floor Planning Web App

Short Description

A browser-based home design application with authenticated project management, a custom 2D floor-plan editor, real-time 3D visualization, auto-save, export workflows, and polished dashboard/authentication screens.

Tech Stack

React, TypeScript, Vite, Tailwind CSS, Zustand, Konva, Three.js, React Three Fiber, FastAPI, MongoDB, Beanie, JWT, Docker.

Key Features

- User authentication.
- Project dashboard.
- 2D wall drawing with snapping.
- Doors, double doors, windows, gates, stairs, furniture, and text.
- Properties inspector.

- Editable wall dimensions and units.
- Undo/redo.
- Auto-save.
- Save as PDF/Word/Image.
- Print.
- Real-time 3D visualization.
- 3D video recording.

Technical Highlights

- Built custom geometry interactions in a canvas editor.
- Converted 2D floor-plan data into 3D scene geometry.
- Designed a nested MongoDB schema for floor-plan persistence.
- Implemented JWT-authenticated API access.
- Added debounced auto-save and undo/redo history.

15. Suggested Resume Version

Use this for resume:

'text

HomePlanner - Full-Stack Floor Planning App

- Built a full-stack browser-based floor planning tool with React, TypeScript, FastAPI, MongoDB, Konva, and Three.js.
- Implemented a custom 2D canvas editor with wall drawing, snapping, doors, windows, stairs, furniture, text labels, selection, dragging, resizing, keyboard shortcuts, undo, and redo.
- Developed real-time 3D visualization by transforming 2D floor-plan geometry into Three.js wall, opening, floor, and furniture meshes.
- Added JWT authentication, protected routes, project CRUD, debounced auto-save, file export, print workflows, and a polished project dashboard.

“

16. Final Assessment

HomePlanner is a strong portfolio project because it demonstrates multiple layers of engineering:

- UI/UX design
- frontend state management
- 2D graphics
- 3D graphics

- backend APIs
- database modeling
- authentication
- persistence
- export workflows
- product iteration

The project is suitable for job applications because it shows that you can build more than a simple CRUD app. It proves you can handle complex user interaction, spatial data, and full-stack architecture.

To make it stronger for hiring, the next best step is to deploy it publicly and add backend-stored project thumbnails. After that, record a short demo video and place the project on your resume, GitHub, and portfolio.